

Pipeline DataOps

NYC Citi Bike Trips

Estudiante:

Sebastian Escobar N.

Profesor:

Felipe Zambrano B.

Contenido

1. Introducción	3
2. Metodología Aplicada	4
3. Planificación del Proyecto	5
3.1 Product Backlog	5
3.2 Carta Gantt — Semanas 5 a 11	5
4. Explicación Técnica del Pipeline	6
4.1 Etapa 1 — Ingesta de Datos	7
Query de extracción	8
4.2 Etapa 2 — Limpieza y Transformación	8
Feature Engineering	8
Filtro de Outliers	8
Eliminación de Data Leakage	8
Persistencia	8
4.3 Etapa 3 — Validación Estructural y Semántica	9
Validación estructural	9
Validación semántica	9
4.4 Etapa 4 — Carga y Visualización	9
Componentes técnicos	9
CI/CD	9
5. Plan de Seguridad – Entorno DataOps	10
5.1 Marco Legal Aplicable	10
5.2 Controles Técnicos Implementados	10
Cifrado en reposo	10
Gestión de credenciales	10
Control de acceso	10
Seguridad del pipeline	10
6. Documentación del Código y Evidencias	11
6.1 Repositorios GitHub	11
6.3 Evidencias de Ejecución AWS Glue	11
7. Estrategia de KPIs de Monitoreo	12
7.1 Implementación de Alertas	12
8. Conclusiones y Próximos pasos	13
8.1 Conclusiones	13
8.2 Próximos Pasos	13

1. Introducción

El presente proyecto académico consiste en el diseño e implementación de un pipeline DataOps completo sobre el dataset público NYC Citi Bike Trips, disponible en Google BigQuery. El objetivo central es demostrar la aplicación de un ciclo de vida de datos desde la ingesta hasta la visualización utilizando servicios en la nube de AWS y buenas prácticas de automatización.

El pipeline implementado sigue una arquitectura de tres capas (Landing → Curated → Consumo) y está compuesto por cuatro etapas funcionales: (1) extracción de una muestra aleatoria desde BigQuery y carga en Amazon S3, (2) transformación y limpieza mediante un Job de AWS Glue Python Shell, (3) validación estructural y semántica realizada durante el EDA en Jupyter, y (4) consumo de los datos curados a través de un dashboard Flask desplegado en Render con integración CI/CD vía GitHub Actions y Docker.

El valor que aporta esta solución a una organización hipotética es la capacidad de transformar datos brutos de sistemas de bicicletas compartidas en información procesada y consumible de manera automatizada, eliminando intervención manual en el proceso de transformación y garantizando reproducibilidad. El costo total estimado del proyecto en la nube se mantiene entre USD 2 y USD 5 para el ciclo completo gracias al uso de servicios serverless.

2. Metodología Aplicada

Para este proyecto se aplica una metodología Adaptativa basada en los principios de Scrum.

La elección de Scrum se justifica por las siguientes razones técnicas y contextuales:

- **Alcance emergente:** al comenzar el proyecto no se tenía certeza del estado ni la calidad del dataset. Era necesario explorar los datos (EDA) antes de definir exactamente qué features construir y qué filtros aplicar, lo que requiere iteraciones cortas.
- **Entregas incrementales:** cada etapa del pipeline (extracción, ETL, dashboard) es un incremento de valor funcional independiente que puede validarse por separado.
- **Adaptación rápida:** el uso de AWS Academy Learner Lab impone restricciones operacionales (sesiones de 4 horas, credenciales rotativas). Scrum permite ajustar la planificación sprint a sprint frente a este tipo de restricción.
- **Trabajo individual:** en contextos de equipo reducido o unipersonal, Scrum simplificado (sin ceremonias formales) es más eficiente que metodologías predictivas que requieren planificación exhaustiva upfront.

No se aplica una metodología Predictiva (cascada/PMBOK clásico) porque el proyecto requiere validar hipótesis sobre los datos antes de diseñar el esquema final de transformación.

3. Planificación del Proyecto

3.1 Product Backlog

- **EP-01 | Ingesta de datos:** Conectar BigQuery, ejecutar query con muestreo RAND(), exportar Parquet a S3 Landing.
- **EP-02 | Transformación ETL:** Job Glue con feature engineering, filtro de outliers, guardado en S3 Curated en formato Snappy Parquet.
- **EP-03 | Análisis EDA:** Notebook Jupyter con análisis de distribución, correlaciones y definición de variable objetivo.
- **EP-04 | Dashboard:** API Flask que lee Parquet desde S3 y presenta resumen de datos vía endpoint / y /health.
- **EP-05 | CI/CD:** Dockerfile + render.yaml + GitHub Actions para despliegue automático en Render.
- **EP-06 | Documentación:** README, docstrings en código, informe técnico, diagrama de arquitectura.

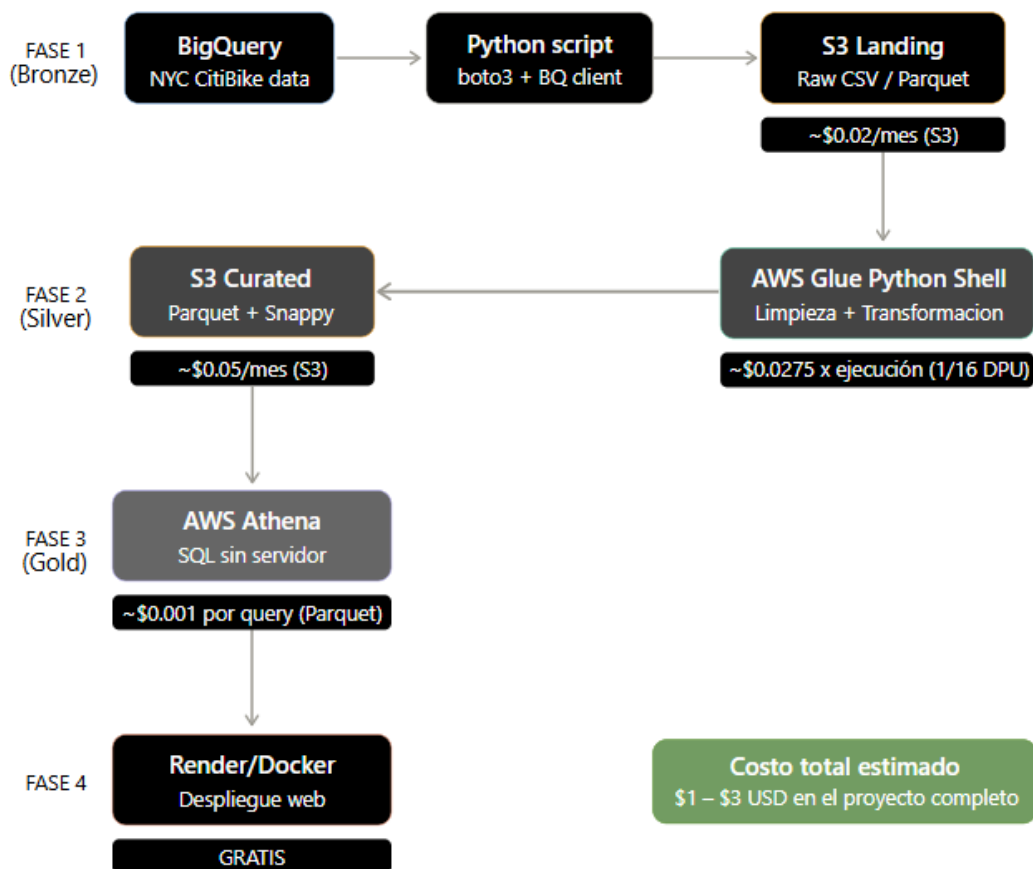
3.2 Carta Gantt — Semanas 5 a 11

Semana	Actividad	Responsable	Entregable
S5	Definición del problema y alcance del proyecto	S. Escobar	Acta de inicio
S6	Extracción de datos desde BigQuery y carga en S3 (Landing)	S. Escobar	Script extract.py + Parquet en S3
S7	Análisis Exploratorio de Datos (EDA)	S. Escobar	Notebook EDA_Citibike_trips.ipynb
S8	Desarrollo ETL con AWS Glue Python Shell	S. Escobar	PipelineETL_AwsGlue.py + S3 Curated
S9	Implementación dashboard Flask + CI/CD en Render	S. Escobar	App desplegada en Render
S10	Validación, pruebas y documentación técnica	S. Escobar	Tests, logs, README
S11	Informe final + presentación + demo funcional	S. Escobar	Informe PDF + demo en vivo

4. Explicación Técnica del Pipeline

El pipeline sigue una arquitectura de medallón con tres zonas de almacenamiento en S3:

- **Landing (Bronze)**: datos crudos en Parquet, tal como se extraen desde BigQuery.
- **Curated (Silver)**: datos transformados, filtrados y enriquecidos con features de ingeniería, en formato Snappy Parquet.
- **Consumo (Gold)**: acceso al dataset curado desde el dashboard Flask vía URL presignada de S3.



Etapa	Herramienta	Descripción	Salida
Ingesta	Python + BigQuery Client + boto3	Extracción de muestra aleatoria (~1%) desde dataset público de BigQuery. Filtros: tripduration > 60s, birth_year no nulo.	citibike_trips.parquet en s3://citibike-datalake-seb/landing/
Transformación	AWS Glue Python Shell (awscli)	Creación de features temporales, cálculo de edad, filtro de outliers (60s–3600s, edad ≤90), eliminación de end_station_id.	citibike_curated.snappy.parquet en s3://.../curated/
Validación	Pandas (describe, isna, duplicated)	EDA confirma 0 nulos, 0 duplicados en 474.491 registros. Percentiles de tripduration y age revisados.	Notebook EDA con estadísticas validadas
Carga / Visualización	Flask + boto3 + Render (Docker)	El dashboard lee el Parquet curado desde S3 vía URL presignada y expone resumen en endpoint /.	Dashboard web activo en Render

4.1 Etapa 1 — Ingesta de Datos

Script: src/extract.py (repositorio Extract-Bigquery-Dataset)

La ingesta se realiza mediante un script Python que ejecuta el siguiente flujo:

- Carga de variables de entorno desde .env (ruta al JSON de credenciales de BigQuery, configuración de AWS).
- Creación del cliente BigQuery usando la librería google-cloud-bigquery autenticado con una Service Account con permisos de solo lectura sobre el dataset bigquery-public-data.new_york_citibike.
- Ejecución de la query con muestreo aleatorio (RAND() < 0.01), filtros de calidad (tripduration entre 60 y 7.200 segundos, birth_year no nulo) y selección de 7 columnas relevantes.
- Guardado del resultado como citibike_trips.parquet en la ruta local ../data/raw/.
- Subida del Parquet a S3 mediante boto3.client('s3').upload_file() al bucket citibike-datalake-seb, prefijo landing/.

El dataset resultante contiene 474.491 registros con las columnas: tripduration, starttime, start_station_id, end_station_id, usertype, gender, birth_year.

Query de extracción

```
SELECT tripduration, starttime, start_station_id, end_station_id,
       usertype, gender, birth_year
FROM `bigquery-public-data.new_york_citibike.citibike_trips`
WHERE tripduration IS NOT NULL
      AND birth_year IS NOT NULL
      AND tripduration > 60
      AND tripduration < 7200
      AND RAND() < 0.01
```

4.2 Etapa 2 — Limpieza y Transformación

Script: docs/PipelineETL_AwsGLue.py (repositorio Extract-Bigquery-Dataset), ejecutado como Job de tipo Python Shell en AWS Glue con 1/16 DPU.

El ETL realiza las siguientes operaciones sobre el dataset Landing:

Feature Engineering

- Conversión de starttime a tipo datetime con `pd.to_datetime(..., errors='coerce')`.
- Extracción de variables temporales: year, month, hour, dayofweek.
- Creación de indicador binario `is_weekend`: 1 si `dayofweek` $\in \{5, 6\}$ (sábado o domingo), 0 el resto.
- Cálculo de la edad aproximada: `age = year - birth_year`.

Filtro de Outliers

- `tripduration > 60` segundos y `< 3.600` segundos (1 hora): elimina registros con duración inválida o extrema.
- `age ≤ 90`: elimina registros con año de nacimiento claramente erróneo (el EDA detectó 246 registros con edad > 90 sobre un `birth_year` mínimo de 1885).

Eliminación de Data Leakage

- Se elimina la columna `end_station_id` porque representa información del futuro (el destino no se conoce al inicio del viaje) y su inclusión causaría fuga de datos en un modelo predictivo posterior.

Persistencia

El DataFrame resultante se guarda en S3 Curated usando `awswrangler.s3.to_parquet()` con compresión Snappy, que reduce el tamaño del archivo manteniendo velocidad de lectura óptima para Athena y Pandas.

```
OUTPUT_PATH = 's3://citibike-datalake-seb/curated/citibike_trips/citibike_curated.snappy.parquet'
```


4.3 Etapa 3 — Validación Estructural y Semántica

Herramienta: Jupyter Notebook (EDA_Citibike_trips.ipynb), ejecutado en Google Colab con el dataset descargado desde Google Drive.

Validación estructural

- **Shape:** 474.491 filas × 7 columnas confirmadas.
- **Tipos de datos:** tripduration e IDs como Int64, starttime como datetime64[us], campos categóricos como object.
- **Nulos:** df.isna().sum() → 0 nulos en todas las columnas.
- **Duplicados:** df.duplicated().sum() → 0 duplicados.

Validación semántica

- **tripduration:** rango [61, 7195] segundos; media 748s (~12 min), mediana 585s. Distribución sesgada a la derecha con outliers en el extremo superior (justifica el filtro >3600s en el ETL).
- **birth_year:** rango [1885, máx]. Se identificaron 246 registros con age > 90, tratados como errores operacionales.
- **Variable objetivo:** se define viaje_largo = 1 si tripduration > 900 segundos (15 min). Distribución: 72.2% viajes cortos, 27.8% largos.
- **Patrones detectados:** tipo de usuario (Customer vs Subscriber), hora del día e indicador de fin de semana muestran diferencias significativas en la proporción de viajes largos, confirmando su utilidad como features predictivos.

4.4 Etapa 4 — Carga y Visualización

El dashboard Flask (repositorio Flask-Dashboard-S3) consume el Parquet curado desde S3 y expone una interfaz web con resumen del dataset.

Componentes técnicos

- app/services/s3_service.py: genera una URL presignada de S3 usando boto3 para acceso temporal sin exponer credenciales.
- app/services/parquet_service.py: descarga el Parquet vía URL presignada con pd.read_parquet() y construye el objeto resumen (total_registros, columnas, preview de 10 filas).
- app/routes.py: Blueprint Flask con dos endpoints GET / (dashboard HTML) y GET /health (estado del servicio).
- Despliegue: imagen Docker basada en python:3.11-slim, expone el puerto 5000, servida por Gunicorn. El render.yaml configura el despliegue automático en Render con las variables de entorno AWS_REGION y las referencias S3.

CI/CD

- El repositorio Flask-Dashboard-S3 cuenta con carpeta .github/workflows/ con pipeline de integración continua.
- El archivo render.yaml define autoDeploy: true, lo que activa el despliegue automático en cada push a la rama main.
- La carpeta tests/ contiene pruebas automatizadas ejecutadas con pytest antes del despliegue.

5. Plan de Seguridad – Entorno DataOps

5.1 Marco Legal Aplicable

- **Ley N°19.628 (Chile) Protección de la Vida Privada:** aunque el dataset es público, establece el marco para el tratamiento de datos personales en proyectos chilenos. El dataset NYC Citi Bike no contiene datos directamente identificables, pero incluye campos sensibles indirectos (género, fecha de nacimiento).
- **Reglamento General de Protección de Datos (GDPR - UE):** referencia de buenas prácticas para minimización de datos; se aplica al eliminar end_station_id (dato no necesario para el modelo) y al no almacenar información personal no requerida.
- **AWS Shared Responsibility Model:** AWS es responsable de la seguridad de la infraestructura física; el estudiante es responsable de la configuración correcta de IAM, S3 y cifrado en tránsito y en reposo.

5.2 Controles Técnicos Implementados

Cifrado en reposo

- **S3 Bucket citibike-datalake-seb:** cifrado SSE-S3 habilitado con Bucket Key (reducción de costos de llamadas KMS). Todos los objetos escritos en Landing y Curated se cifran automáticamente con AES-256.

Gestión de credenciales

- **Credenciales AWS:** gestionadas mediante AWS CLI (~/.aws/credentials) con session tokens temporales (duración máxima 4 horas, correspondiente al período del Learner Lab). No se almacenan en el repositorio.
- **Service Account BigQuery:** archivo JSON con permisos de solo lectura (roles/bigquery.dataViewer) sobre el dataset público. Referenciado vía variable de entorno GOOGLE_APPLICATION_CREDENTIALS. Excluido del repositorio público mediante .gitignore.
- **Variables de entorno Flask:** configuradas en Render como secretos (sync: false en render.yaml), nunca expuestas en el código fuente.

Control de acceso

- **IAM Role de Learner Lab:** permisos acotados por la política de AWS Academy. El acceso a S3 opera bajo el rol de sesión temporal.
- **.gitignore en ambos repositorios:** excluye .aws/credentials, .env, *.json de service accounts y directorios data/ con archivos Parquet locales.

Seguridad del pipeline

- El endpoint /health expone únicamente estado operacional, sin datos del dataset.
- La URL presignada de S3 tiene tiempo de expiración configurado, limitando el acceso temporal al Parquet curado.

6. Documentación del Código y Evidencias

6.1 Repositorios GitHub

Extracción + EDA + ETL: <https://github.com/Sebithaz-dev/Extract-Bigquery-Dataset>

github.com/Sebithaz-dev/Extract-Bigquery-Dataset

Contiene: src/extract.py, docs/PipelineETL_AwsGLue.py, docs/EDA_Citibike_trips.ipynb, diagrama de arquitectura, .env.example, requirements.txt, .gitignore.

Dashboard Flask + CI/CD: <https://github.com/Sebithaz-dev/Flask-Dashboard-S3>

github.com/Sebithaz-dev/Flask-Dashboard-S3

Contiene: app/ (routes.py, services/), tests/, Dockerfile, docker-compose.yml, render.yaml, requirements.txt, .github/workflows/.

6.3 Evidencias de Ejecución AWS Glue

El Job PipelineETL_AwsGLue.py fue ejecutado exitosamente en AWS Glue Python Shell con las siguientes características:

- **Tipo de Job:** Python Shell (1/16 DPU).
- **Estado final:** Succeeded.
- **Logs de salida confirmados:**

```
[!] Shape original: (474491, 7)
[!] Shape despues del filtro: (N, 12)
[!] Dataset curated guardado en: s3://citibike-datalake-seb/curated/...
```

- El script imprime forma del DataFrame antes y después de los filtros, permitiendo auditar la reducción de registros en cada ejecución.

7. Estrategia de KPIs de Monitoreo

El sistema de monitoreo del pipeline está orientado a detectar degradaciones en tres dimensiones: calidad de los datos, rendimiento de los procesos y disponibilidad del servicio de consumo. A continuación, se presentan los KPIs críticos con sus umbrales y mecanismos de alerta.

KPI	Descripción	Umbral aceptable	Alerta
Tasa de éxito del Job Glue	% de ejecuciones que terminan en estado Succeeded	$\geq 95\%$	$< 90\%$
Filas procesadas (completitud)	Registros salida / registros entrada tras filtros	$\geq 50\%$	$< 30\%$
Latencia ETL (min)	Tiempo de ejecución del Job Glue Python Shell	≤ 5 min	> 10 min
Tamaño Parquet curado (MB)	Peso del archivo en S3 Curated	Estable $\pm 10\%$	Variación $> 25\%$
Tiempo de respuesta API Flask	Latencia del endpoint principal en Render	≤ 2 s	> 5 s
Disponibilidad del dashboard	Uptime del servicio en Render	$\geq 99\%$	$< 95\%$
Cobertura de tests (pytest)	% de líneas cubiertas por tests automatizados	$\geq 70\%$	$< 50\%$

7.1 Implementación de Alertas

- **AWS Glue - CloudWatch Logs:** los logs del Job (stdout del script) registran el shape original y el shape post-filtro. Una variación superior al 25% en el ratio de registros procesados activa revisión manual.
- **AWS S3 - Storage Metrics:** el tamaño del objeto curado en S3 puede monitorearse vía CloudWatch S3 Storage Lens para detectar variaciones anómalas.
- **Render - Health Check:** el endpoint GET /health retorna JSON {"status": "ok"} y está configurado como health check nativo en Render, que reinicia el servicio automáticamente si falla.
- **GitHub Actions - Estado del pipeline CI:** el resultado (pass/fail) de los tests pytest queda registrado en la pestaña Actions del repositorio, con notificación por email ante fallos.

8. Conclusiones y Próximos pasos

8.1 Conclusiones

El proyecto logró implementar un pipeline DataOps funcional end-to-end sobre el dataset NYC Citi Bike Trips, cubriendo todas las etapas del ciclo de vida de los datos: ingesta, transformación, validación, carga y visualización. Los principales logros técnicos son:

- Automatización completa de la extracción desde BigQuery a S3 mediante un script parametrizado con manejo de errores en cada etapa.
- ETL en la nube con AWS Glue Python Shell que produce un dataset curado y libre de outliers en formato optimizado (Snappy Parquet).
- EDA que validó la calidad del dataset y fundamentó las decisiones de feature engineering y definición de la variable objetivo.
- Dashboard Flask con CI/CD automatizado, contenedorizado con Docker y desplegado en producción con costo USD 0 en la capa gratuita de Render.
- Seguridad implementada en múltiples capas: cifrado SSE-S3, credenciales fuera del repositorio, acceso IAM con mínimos privilegios.
- Optimización de costos en la nube, logrando ejecutar todo el pipeline con un costo aproximado de solo USD 2 en AWS, correspondiente principalmente al almacenamiento en S3 y al procesamiento con AWS Glue Python Shell utilizando configuración de 1/16 DPU.

8.2 Próximos Pasos

- Modelo de clasificación: implementar el modelo de Regresión Logística identificado en el EDA para predecir viaje_largo usando los features disponibles al inicio del viaje.
- Automatización del Job Glue: configurar un trigger temporal (ej. semanal) en Glue para re-ejecutar el ETL con nuevas muestras del dataset BigQuery.
- Enriquecimiento del dashboard: agregar gráficos de distribución de tripduration, mapa de estaciones y filtros interactivos utilizando Plotly o Chart.js en el frontend Flask.
- Tests de calidad de datos: incorporar Great Expectations o validaciones Pandas en el ETL para generar reportes automáticos de calidad post-transformación.
- Migración a GitHub Actions completa: incluir el paso de subida a S3 dentro del pipeline CI/CD para que un push a main desencadene automáticamente la extracción y el ETL.